样题5

项目一：平台搭建与运维

(一)任务一：大数据平台搭建

1.子任务一：基础环境准备

(1)`hostnamectl set-hostname master`
`vim /etc/hosts`

```
ip master
ip node01
ip node02
```

`ping -c 4 node01`
(2)`tar -zxvf /opt/software/jdk-8u391-linuxx64.tar.gz -C /opt/module`
`ll /opt/module`
(3)`vim /etc/profile`

```
export JAVA_HOME=/opt/modul/jdk
export PATH=$PATH:$JAVA_HOME/bin
```

(4)`systemctl stop firewalld`
`systemctl disable firewalld`
`systemctl status firewalld`
(5)`ssh-keygen -t rsa`
`ssh-copy-id root@node01`
`systemctl restart sshd`
`ssh node01(node02)`
(6)`scp -r /opt/module/jdk root@node01:/opt/module`
(7)`java -version`
(8)`useradd hadoop`
`passwd hadoop`
`sudo vim /etc/sudoers`

```
hadoop ALL=(ALL)    NOPASSWD:ALL
```

`grep 'hadoop' /etc/sudoers`

2.子任务二：Hadoop 完全分布式安装配置

(1)`tar -zxvf /opt/software/hadoop-3.1.3.tar.gz -C /opt/module`

(2)`mv /opt/module/hadoop-3.1.3 /opt/module/hadoop`

`chown -R hadoop:hadoop /opt/module/hadoop`

(3)`cd /opt/module/hadoop/etc/hadoop`

hadoop-env.sh

```shell
export JAVA_HOME=/opt/module/jdk
export HADOOP_HOME=/opt/module/hadoop
export HADOOP_CONF_DIR=${HADOOP_HOME}/etc/conf
export HADOOP_LOG_DIR=${HADOOP_HOME}/logs
```

core-site.xml

```xml
<property>
   <name>fs.defaultFS</name>
   <value>hdfs://master:9820</value>
</property>
<property>
   <name>hadoop.tmp.dir</name>
   <value>/opt/module/hadoop/tmp</value>
</property>
```

hdfs-site.xml

```xml
<property>
   <name>dfs.replication</name>
   <value>3</value>
</property>
<property>
   <name>dfs.namenode.secondary.http-address</name>
   <value>node01:9868</value>
</property>
<property>
   <name>dfs.datandoe.data.dir</name>
   <value>/opt/module/hadoop/datanode/data</value>
</property>
```

mapred-site.xml

```xml
<property>
   <name>mapreduce.framework.name</name>
   <value>yarn</value>
</property>
<property>
   <name>mapreduce.jobhistory.address</name>
   <value>node01:10020</value>
</property>
<property>
   <name>mapreduce.jobhistory.webapp.address</name>
   <value>node01:19888</value>
</property>
```

yarn-site.xml

```xml
<property>
   <name>yarn.resourcemanager.hostname</name>
   <value>master</value>
</property>
<property>
   <name>yarn.nodemanager.aux-services</name>
   <value>mapreduce_shuffle</value>
</property>
<property>
   <name>yarn.log-aggregation-enable</name>
   <value>true</value>
</property>
<property>
   <name>yarn.log.server.url</name>
   <value>node01:19888</value>
</property>
```

workers

```
master
node01
node02
```

(4) `vim /etc/profile`

```
export HADOOP_HOME=/opt/module/hadoop
export PATH=$PATH:$HADOOP_HOME/bin
```

(5)`scp -r /opt/module/hadoop hadoop@node01:/opt/module`
`chown -R hadoop:hadoop /opt/module/hadoop`
(6)`hdfs namenode -format`
(7)`start-dfs.sh`
`start-yarn.sh`
`mapred --daemon start historysever`
`jps`
(8)`ssh node01`
`jps`

(二)任务二：数据库服务器的安装与运维

1.子任务一：MySQL 安装配置

(1)`tar -zxvf /opt/software/mysql-5.7.251.el7.x86_64.rpm-bundle.tar -C /opt/module`
(2)`rpm -ivh mysql-community-common mysql-community-libs mysql-community-libs-compat mysql-community-client mysql-community-server`
(3)`systemctl start mysqld`
`mysqld --initialize --user=mysql`

2.子任务二：MySQL 运维

(1)`vim /etc/my.cnf` 修改 MySQL 配置文件启用远程连接

```
[mysqld]
bind-address = 0.0.0.0
```

`systemctl restart mysqld`
(2)启用 MySQL 慢查询日志功能，设置慢查询阈值为 2 秒

```
[mysqld]
slow_query_log = ON
long_query_time = 2
slow_query_log_file = /var/log/mysql/slow.log
```

`systemctl restart mysqld`
`show variables like 'slow_query_log';`
`show variables like 'long_query_time';` 验证配置结果
(3)创建名为 hadoop 的数据库用户，密码设置为 Hadoop@2025

```
 create user 'hadoop'@'%' identified by 'Hadoop@2025';
```

(4)为 hadoop 用户授予 bigdata 数据库的 SELECT、INSERT 和 UPDATE 权限

```
grant select,insert,update on bigdata.* to 'hadoop'@'%';
flush privileges;
```

(5)创建名为 monitor 的监控专用用户，仅授予 PROCESS 和 SHOW DATABASES 权限

```
create user 'monitor'@'%' identified by 'Monitor@2025';
grant process on *.* to 'monitor'@'%';
grant show databases on *.* to 'monitor'@'%';
flush privileges;
```

(6)使用 mysqldump 工具备份 bigdata 数据库，将备份文件保存为 bigdata_backup.sql

```SHELL
mysqldump -u root -p bigdata > /opt/backup/bigdata_backup.sql
```

(7)配置数据库字符集为 utf8mb4，并将默认排序规则设置为 utf8mb4_unicode_ci

```
[mysqld]
character-set-server = utf8mb4
collation-server = utf8mb4_unicode_ci
```

`systemctl restart mysqld`
`show variables like 'character_set_server';`
`show variables like 'collation_server';` 验证配置结果

(8)

```SHELL
#!/bin/bash
user="root"
passwd="zzz"
backup='/.../../..'
date=$(date +%Y%m%d_%H%M%S)
database="bigdata"

mkdir -p "$backup"

mysqldump -u"$user" -p"$passwd" "$database" >
"${backup}/${database}_backup_$date.sql"

find "$backup" -name "${database}_backup_*.sql" -mtime +7 -delete
```

`chmod +x bigdata_backup.sh`
`crontab -l`
`(crontab -l 2>/dev/null; echo "0 2 * * * /.../bigdata_backup.sh") | crontab -`

3.子任务三：数据表的创建及维护

`use bigdata`

(1)

```
create table hotel(
   id int primary key auto_increment comment '酒店编号',
   hotel_name varchar(255) comment '酒店名称',
   city varchar(255) comment '城市',
   province varchar(255) comment '省份',
   level varchar(255) comment '星级',
   room_num int comment '房间数',
   score double comment '评分',
   commnet_num varchar(255) comment '评论数'
);
```

(2)

```
use bigdata;
create table comment(
   id int primary key auto_increment comment '评论编号',
   name varchar(255) comment '酒店名称',
   commentator varchar(255) comment '评论人',
   score double comment '评分',
   comment_time datetime comment '评论时间',
   content varchar(255) comment '评论内容'
);
```

(3)

```
update hotel h
join (select province, avg(score) as avg_score from hotel group by province) a
on h.province = a.province
set h.score = least(h.score + 0.5, 5)
where h.score < a.avg_score;
select province, avg(score) from hotel group by province;
```

```
select city, hotel_name, score, commnet_num
from hotel h
where commnet_num = (
    select max(commnet_num) from hotel h2 where h2.city = h.city
);
```

```
select c.commentator, c.comment_time, c.content, c.name
from comment c
where c.commentator in (
    select commentator from comment where score <= 2 group by commentator having
count(*) >= 3
)
order by c.comment_time desc;
```

项目二：数据获取与处理

(一)任务一：数据获取与清洗

1.子任务一：数据获取

有一份出行订单数据文件：订单 ID、日期、乘客年龄段、出行时长（小时）、订单金额（元）、服务评分（1-5 分）、出行方式（如快车、专车）、天气状况、道路状况等信息，数据已存入到 travel_order_data.csv 文件中，请使用 pandas 读取 travel_order_data.csv 并将数据集的前 10 行打印在 IDE 终端

```
import pandas as pd
data = pd.read_csv('travel_order_data.csv',nrows=10)
print(data)
```

2.子任务二：使用 Python 进行数据清洗

(1)删除天气状况为空或为无效值的记录，并将结果存储为 `cleaned_data_c1_74.csv`，N 为删除的数据条数
(2)删除出行时长异常（超过 12 小时或小于 0.1 小时）的记录，并将结果存储为 `cleaned_data_c2_12.csv`，N 为删除的数据条数
(3)将出行方式为空的记录填充为"未知"，并存储为 `cleaned_data_c3_14.csv`，N 为修改的数据条数
(4)将服务评分超出范围（1-5 分）的记录删除，并将结果存储为 `cleaned_data_c4_11.csv`，N 为删除的数据条数
(5)将订单金额明显异常（超过该出行方式平均订单金额 5 倍）的记录删除，将结果存储为 `cleaned_data_c5_5.csv`，N 为删除的数据条数

(二)任务二：数据标注

1.子任务一：乘客价值分类标注

使用 Python 编写脚本，根据乘客的订单金额和出行时段对乘客进行价值分类标注。在数据集中新增一列"乘客价值类型"，根据上述标准进行标注，存入 passenger_value_mark.csv 文件中

(1)高价值乘客：订单金额高于平均值的 1.5 倍且出行时段位于高峰时段（如 07-09 点或 17-19 点）
(2)中价值乘客：订单金额在平均值的 0.5-1.5 倍之间且出行时段在平峰时段（如 10-16 点）
(3)低价值乘客：不满足上述条件的乘客
标注结果共 198 条，其中高价值乘客 7 单、中价值乘客 27 单、低价值乘客 164 单，高价值乘客占比最低，仅为 3.54%，绝大多数订单属于低价值乘客

```
编号 出行方式 订单金额(元) 出行时段(小时) 乘客价值类型
1 未知 34.72 14.9 低价值乘客
2 拼车 96.97 19.4 低价值乘客
3 未知 18.53 4.9 低价值乘客
6 拼车 100.23 14.3 中价值乘客
```

2.子任务二：出行需求旺季标注

使用 Python 编写脚本，根据每月的订单总量和平均订单金额对出行需求强度进行标注。在数据集中新增一列"需求季节类型"，根据上述标准对每个月份进行标注，存入 demand_season_mark.csv 文件中
(1)旺季：月订单量超过年平均值的 120% 且月平均订单金额超过年平均值的 110%
(2)平季：月订单量在年平均值的 80%-120% 之间或月平均订单金额在年平均值的 90%-110% 之间
(3)淡季：月订单量低于年平均值的 80% 且月平均订单金额低于年平均值的 90%
标注结果共 12 个月，其中平季 9 个月、淡季 2 个月（2 月订单量 13 单、均价 52.62 元、同比 74%；6 月订单量 13 单、均价 61.22 元、同比 86%）、旺季 1 个月（8 月订单量 26 单、均价 95.26 元、同比 133%），8 月是全年唯一的需求旺季，9 月订单量虽达 30 单为全年最高，但均价仅 71.65 元（同比 100%），故判为平季

```
月份 订单总量 平均订单金额(元) 同比年平均 需求季节类型
2月 13 52.62 74% 淡季
6月 13 61.22 86% 淡季
8月 26 95.26 133% 旺季
9月 30 71.65 100% 平季
```

(三)任务三：数据统计

1.子任务一：HDFS 文件操作

(1)`hdfs dfs -mkdir /travel_order_data` 在 HDFS 目录下新建目录
(2)`hdfs dfs -chmod 777 /travel_order_data` 修改权限，赋予目录 /travel_order_data 最高 777 权限
(3)`hdfs dfs -put travel_order_data.csv /travel_order_data` 将本地的出行订单数据集上传至

HDFS 的 /travel_order_data 目录下
`hdfs dfs -ls -h /travel_order_data` 验证文件是否上传成功

2.子任务二：使用 MapReduce 进行数据分析

(1)编写 MapReduce 程序，统计不同出行方式的日均订单数量
(2)将统计结果存储到 HDFS 的 `/travel_analysis/daily_avg_orders` 目录下
(3)运行 MapReduce 程序并查看执行结果，将运行命令和结果的前十行截图粘贴至【提交结果.docx】中对应的任务序号下

```
出行方式 日均订单数
豪华车 0.3548
快车 0.3226
拼车 0.2968
专车 0.2129
未知 0.0903
```

本数据集覆盖 2025-01-01 至 2025-12-30 共 155 个自然日，198 单合计日均 1.2774 单/天。各出行方式的有单天数分别为豪华车 53 天、快车 44 天、拼车 44 天、专车 32 天、未知 14 天。

3.子任务三：使用 Hive 进行数据分析

(1)在 Hive 中创建外部表 travel_orders，加载 HDFS 中的出行订单数据

```SQL
create external table travel_orders(
    订单ID string,
    日期 string,
    乘客年龄段 string,
    `出行时长(小时)` double,
    `订单金额(元)` double,
    服务评分 double,
    出行方式 string,
    天气状况 string,
    道路状况 string,
    `出行时段(小时)` double
)
row format delimited
fields terminated by ','
location '/travel_order_data';
```

`load table travel_orders;` 加载 HDFS 中的出行订单数据
(2)编写 HQL 查询，分析以下内容
统计每个月的订单总量和平均订单金额

```sql
select substr(日期,1,7) as 月份,
       count(*) as 订单总量,
       round(avg(`订单金额(元)`),2) as 平均订单金额
from travel_orders
group by substr(日期,1,7);
```

计算不同天气状况下的订单总量和平均服务评分

```sql
select 天气状况,
       count(*) as 订单总量,
       round(avg(服务评分),2) as 平均服务评分
from travel_orders
group by 天气状况;
```

统计不同出行方式在工作日和周末的订单分布

```sql
select 出行方式,
       case when dayofweek(cast(日期 as date)) in (1,7) then '周末' else '工作日' end
as 日期类型,
       count(*) as 订单分布
from travel_orders
group by 出行方式, case when dayofweek(cast(日期 as date)) in (1,7) then '周末' else
'工作日' end;
```

(3)将查询结果导出到本地文件系统

```sql
insert overwrite local directory '/opt/hive_export/monthly'
select substr(日期,1,7) as 月份,
       count(*) as 订单总量,
       round(avg(`订单金额(元)`),2) as 平均订单金额
from travel_orders
group by substr(日期,1,7);
```

项目三：业务分析与可视化

(一)任务一：数据分析与可视化

1.子任务一：数据分析

(1)分析各出行方式的订单分布，找出最主要的出行方式（占比超过 40%）：共 5 种出行方式，豪华车 55 单占 27.78%、快车 50 单占 25.25%、拼车 46 单占 23.23%、专车 33 单占 16.67%、未知 14 单占 7.07%，**没有占比超过 40% 的出行方式**，订单在快车、拼车、豪华车之

间较为均衡地分散
(2)计算不同年龄段乘客的平均订单金额和常用出行时段，发现最具消费潜力的乘客群体：46-55 岁群体平均订单金额 78.77 元为最高（常用时段 11 时、40 单），55 岁以上 77.17 元（11 时、40 单）、36-45 岁 72.62 元（4 时、33 单）、26-35 岁 71.70 元（15 时、44 单）、18-25 岁 65.66 元最低（19 时、41 单），年长乘客客单价更高而年轻乘客下单更集中在晚高峰
(3)分析周末和工作日的订单量差异，计算周末订单量相对工作日的增长百分比：工作日 145 单、周末 53 单，周末订单量相对工作日增长 -63.45%，周末出行需求明显收缩
(4)统计不同天气状况的订单比例，并根据平均订单金额对天气状况进行排序：大雨 24 单占 12.12%（均价 81.16 元）均价最高，雪 6 单占 3.03%（77.53 元），多云 49 单占 24.75%（73.90 元），晴 73 单占 36.87%（73.49 元）比例最高，小雨 31 单占 15.66%（69.28 元），雾 13 单占 6.57%（67.64 元），未知 2 单占 1.01%（27.46 元）均价最低。恶劣天气虽然订单占比不高，但单均金额明显高于晴天，恶劣天气溢价效应显著
(5)分析服务评分与订单金额的关系，计算相关系数：相关系数为 0.1776，呈极弱的正相关，说明服务评分对订单金额的拉动作用有限，乘客下单决策更多受出行方式、时段和天气等价格与供需因素驱动

2.子任务二：数据可视化

(1)创建玫瑰图展示各出行方式订单量分布，使用玫瑰图展示不同出行方式的年度订单总量占比，每个扇形区域的半径表示订单量大小，颜色深浅代表订单量级别，并添加图例说明和标题
(2)制作气泡图分析订单特征，横轴表示平均订单金额，纵轴表示日均订单量，气泡大小表示平均服务评分，颜色表示出行方式类型
(3)绘制堆叠面积图展示一周内各时段订单量变化趋势，使用堆叠面积图展示一周内（周一至周日）各小时段订单量的变化趋势，不同颜色代表不同日期类型（工作日/周末），需要显示小时数据点，并支持区域高亮显示
(4)创建雷达图对比不同乘客群体特征，使用雷达图对比不同年龄段乘客的出行特征，包含订单金额、出行距离偏好、服务评分、出行频率、夜间订单比例等维度，并添加数据标签

(二)任务二：业务分析

1.基础指标分析

(1)运力指标分析：日均订单量为 1.28 单/日；订单波动系数（标准差/平均值）为 0.4235，明显高于房地产交易场景，说明出行订单的波动性更大；节假日期间日均订单量 1.33 单/日，普通日期日均订单量 1.28 单/日，节假日订单比为 1.0456，节假日需求略有上浮
(2)收益行为分析：平均客单价为 73.14 元/单（总金额 14480.80 元 / 198 单）。不同群体消费差异显著，例如 18-25 岁群体中豪华车均价 79.98 元、专车仅 53.12 元；26-35 岁专车 69.77 元；36-45 岁拼车 82.87 元最高、未知仅 57.32 元；46-55 岁豪华车 114.83 元最高、未知 43.61 元最低；55 岁以上快车 97.82 元为全表最高，未知 88.44 元次之
(3)服务质量评估：服务评分采用 5 分制，198 条订单全部为有效评分，各评分档位分布为 1 分 40 单、2 分 38 单、3 分 31 单、4 分 47 单、5 分 42 单，加权平均值（按订单量加权）为 3.07

分，整体服务质量处于中等偏下水平。分出行方式看，拼车 3.15 分最高，豪华车 3.09 分，专车 3.06 分，快车 3.02 分，未知 2.86 分最低

2.运营建议

(1)动态定价策略：结合订单波动规律和乘客支付能力，设计分时段、分区域的动态调价方案。数据表明恶劣天气客单价明显高于晴天（大雨 81.16 元 vs 晴天 73.49 元），46-55 岁群体客单价达 78.77 元，可在恶劣天气与该群体场景上设置溢价
(2)运力调度优化：针对高峰时段和热点区域，提出司机调度的具体优化措施。周末订单量仅为工作日的 36.55%（-63.45%），而 36-45 岁群体常用出行时段集中在凌晨 4 时、18-25 岁集中在晚高峰 19 时，应按小时段差异化投放运力，避免工作日运力空转
(3)精准营销建议：基于高价值乘客画像，设计有针对性的优惠券或会员权益策略。高价值乘客仅 7 单且集中在金额高于均值 1.5 倍的时段，夜间的未知方式订单均价也达 88.44 元，建议面向高金额、高价值时段订单定向发放优惠券并沉淀会员权益

项目四：AI 大模型应用开发综合实践

(一)任务一：AI 大模型服务综合部署

1.子任务一：基本配置

本任务需要使用 root 用户完成相关配置，已提供操作系统镜像及需要配置的前置环境，所需软件安装包均在 /root/ 目录下
(1)`mkdir -p /media/cdrom` 在 AI-SRV 中将操作系统镜像挂载至 /media/cdrom，并实现开机自启
情况一：镜像文件在 `/root/` 目录下

```
mount -o loop /root/openEuler-22.03-LTS-SP4-x86_64-dvd.iso /media/cdrom
```

情况二：镜像已通过虚拟光驱加载到设备 `/dev/sr0`

```
mount /dev/sr0 /media/cdrom
```

编辑 `vim /etc/fstab`

```
末尾添加
/dev/sr0    /media/cdrom    iso9660    defaults,ro    0    0
```

注意：
`/dev/sr0` 光驱设备名（用 `lsblk` 确认）
`/media/cdrom` 挂载点
`iso9660` 光盘文件系统类型

`defaults,ro` 表示默认选项且只读（推荐加 `ro` 以避免警告）
`0 0` 表示不备份、不检查
`mount -a` 验证
如果没有报错，说明配置成功
(2)`vim /etc/yum.repos.d/local.repo` 在 AI-SRV 中完成软件源配置文件的编写，禁用 GPG 校验，并为源数据建立缓存

```
[local]
name=Local Repository
baseurl=file:///media/cdrom
enabled=1
gpgcheck=0
```

`yum makecache` 建立缓存

```
(可能需要删除所有默认远程源)
cd /etc/yum.repos.d/
ls 查看是否有默认远程源
rm -f /etc/yum.repos.d/openEuler.repo 删除除了local.repo以外的
```

验证是否仅使用本地源

2.子任务二：环境搭建

本任务需要使用 root 用户完成相关配置，已提供 ollama 安装包及需要配置的前置环境
(1)`groupadd ollama` 为 ollama 配置用户/组，该用户仅用于运行服务，禁止其登录系统，家目录位于 /usr/share/ollama
`useradd -r -g ollama -d /usr/share/ollama -s /sbin/nologin ollama`
(2)`usermod -aG ollama root` 为 root 用户加入附加组 ollama
(3)`tar -zxvf /root/ollama-linux-amd64.tgz -C /usr` 将 ollama-linux-amd64.tgz 解压安装至 /usr 目录下
(4)编写 SystemD 服务配置 `vim /etc/systemd/system/ollama.service`，使用服务运行专用用户完成启动，无论发生任何意外情况，服务都会进行自动重启（冷却 3 秒），以保证服务可靠运行，服务需要监听所有地址

```
[Unit]
Description=Ollama Service
After=network-online.target

[Service]
ExecStart=/usr/bin/ollama serve
User=ollama
Group=ollama
Restart=always
RestartSec=3
Environment="OLLAMA_HOST=0.0.0.0:11434"

[Install]
WantedBy=multi-user.target
```

`systemctl daemon-reload` 重载服务配置
(5)`systemctl enable ollama` 启动 ollama 服务并完成开机启动配置
`systemctl start ollama`
`systemctl status ollama` 查看服务状态，确认端口监听正常
(6)`ollama create qwen2.5:1.5b -f 1.5b.tgz` 导入 1.5b.tgz 压缩文件模型至 ollama
`ollama list` 查看已导入的模型列表

(二)任务二：AI 大模型综合应用开发

1.子任务一：Dify 运行环境搭建

本任务需要使用 root 用户完成相关配置，已提供安装 dify 及需要配置的前置环境
(1)`yum install -y docker` 在 AI-SRV 中完成 Docker 环境的安装
`systemctl start docker`
`systemctl enable docker`
`docker --version` 查看 Docker 版本信息
(2)`tar -zxvf /root/dify-0.14.2.tar.gz -C /opt` 将 dify-0.14.2.tar.gz 解压至 /opt 路径中
`cd /opt/dify/docker` 进入 dify 的 docker 目录
(3)`docker load -i /opt/dify/docker/dify-images/*.tar` 完成 dify 镜像的导入，相关文件位于 dify-images 目录中
`docker images` 检查镜像列表
(4)`docker compose up -d` 完成 dify 的启动，并在客户机中访问 AI-SRV 中的 dify web 服务
`docker ps` 查看容器运行状态
(5)在客户机中访问 dify，完成用户注册，邮箱为 admin@dify.ai，用户名为 admin，密码为 Qwer1234，注册后登录 dify 管理页面
(6)在客户机中访问 dify，完成对 ollama 模型提供商的注册，API 地址填 `http://<AI-SRV 的 IP>:11434`，模型名称选已导入的 qwen2.5:1.5b

2.子任务二：出行服务智能助手构建

(1)根据出行服务数据分析场景需要，创建空白应用 "Travel Service Assistant"
(2)为用户提供出行服务数据分析智能查询功能，其中参数包含 Query（查询内容），支持以下类型的查询
出行方式查询（如："查询'快车'在 2025 年的月度订单总量"）
乘客画像分析（如："查询周末订单金额最高的年龄段"）
服务质量对比（如："对比晴天与雨天专车与快车的服务评分差异"）
出行趋势预测建议（基于历史数据的简单推演）
(3)为确保结果准确性，需要从以下多个维度提供数据支持
基于 MySQL 数据库中的结构化出行订单数据（如 bigdata 数据库中的 travel_order、hotel 等表）
基于清洗后的 CSV 文件（如 cleaned_data_c5_N.csv 及相关处理结果）
基于大模型的智能解析和推理能力
(4)配置提示词要求
系统角色设定为"出行服务数据分析专家"
支持 SQL 查询语句的自动生成和解释
支持自然语言到出行服务数据分析需求的转换
输出结果需包含数据来源说明（如数据库表、CSV 文件等）
(5)运行应用，测试以下查询场景
"查询 2025 年第三季度'快车'出行方式的月度订单总量"
"哪些年龄段的乘客在周末订单金额最高？"
"对比晴天与雨天情况下，'专车'与'快车'的平均服务评分差异"
"预测下个月工作日早高峰（7-9 点）的订单趋势（基于历史同期数据）"
"查询 2025 年节假日期间订单量最高的前三个城市及其平均订单金额"